CNS: CSR: Small: Collaborative Research: Creating Realistic Synthetic I/O Workloads Through Generative Machine Learning

A major challenge for system architects and performance analysts is identifying and obtaining workloads to evaluate and compare system designs. For example, CERN collects approximately *86 TB of data each day* from its Large Hadron Collider; 90% is filtered out [29], the remaining 8 TB is stored to monitor and optimize the global CERN storage framework. Being able to generate realistic synthetic traces would enable CERN to filter out almost all of this remaining daily 8 TB, reducing their cost, power, and computer overhead [29].

There are two key gaps in trace generation that this proposal addresses: 1) current synthetic traces do not capture higher order trace properties [8], and 2) there is no standard, quantitative way to validate the representativeness of synthetic traces [4]. The PIs propose a suite of machine learning tools to **generate and validate realistic workloads**, allowing users to make storage decisions without the *privacy*, *cost*, and *management* overheads of storing raw I/O traces.

The project is composed of three mutually reinforcing **research objectives**:

1. ***Generate globally representative synthetic traces.*** The PIs propose to pioneer the use of generative machine learning models to generate synthetic trace data. The workloads generated will be representative enough to negate the need to generate different traces to test different system components.
2. ***Develop test frameworks to judge representativity of generated traces.*** The PIs will build a machine learning based statistical analysis module to test the similarity of traces generated by this project and their corresponding real traces. This customized validation framework will ensure the toolsuite is useful and verifiable to users in the absence of an in-house ML expert.
3. ***Capturing and emulating inconsistencies in real workloads.*** To realistically emulate a time series with higher order effects, the PIs must model outliers. The PIs will investigate generative model issues caused by bursts and anomalies in training data, such as overfitting, diminished gradient and modal collapse, with the ultimate goal of emulating these inconsistencies in generated workloads.

Intellectual Merit This project will transform synthetic trace generation in storage and generalize generative models to represent an entire trace instead of a facet [16, 83]. This will pave the way for inter-disciplinary research in statistics, physics, systems, and performance testing. This research will also reduce provisioning and operational cost for exascale storage, saving both cost and power for both government and industrial infrastructure. Finally, the validation framework will allow these models to be broadly applied without any specialized local knowledge.

Broader Impacts: Synthetic traces are the best way to anonymize usage data. Having representative synthetic trace data will facilitate trusted collaboration between system vendors and consumers since most privacy concerns are mitigated if no real data is shared. Healthcare industries in particular require a high privacy standard, making it almost impossible to generate representative real workloads. This work will allow health care infrastructure to be externally analyzed without the overhead of HIPPA data management for the first time.

The PIs will develop course modules for generative models that will attract students from the statistics, physics and systems community. Wildani is also working with two undergraduate URMs in developing an open, general purpose code-base for learning about GANs from a system research perspective. **Key Words: Storage Systems; Synthetic trace generation; Machine Learning; Generative Adversarial Networks; Privacy**

A Introduction

Dynamic, fine-grained tuning and optimization is critical to building storage systems that are cost and power efficient [13]. This tuning requires a precise and accurate understanding of how users are interacting with a system along with, ideally, some ability to project how consistent this behaviour will remain over a period of time [32]. Organizations like IBM, Samsung, and CERN anticipate radical changes in computing and software [29], which require workloads representative of the real consumer environment to test their systems before shipping them to consumers. Collecting and disseminating real user workloads to track storage system usage incurs storage and performance overheads. It requires active management for sampling, and has the potential to leak private user and corporate data [67], which makes consumers skeptical about sharing their workloads. Duplication of real traces may be used as an alternative, but are limited by laws against administrators tracing systems. Additionally, larger traces have a higher storage overhead and lack representation of holistic workload characteristics, making it difficult to achieve statistical confidence in results [39].

The alternative to real traces is synthesized workloads that capture the long term trends as well as the local features that indicate actionable changes in system usage. Current synthetic workload generators such as FileBench or IOmeter, are designed to have controlled repeatable results for accurate comparisons and minimal bottlenecks across different isolated tests [75, 36]. While these workloads provide a usable benchmark for testing individual system components, they lack the complexity to test the system trends needed for tuning [51, 17, 74, 118, 106, 25, 117, 12, 46, 2, 9, 23]. As a compromise, companies either run smaller workloads focusing on a subset of system characteristics, providing them with an incomplete picture of the system's performance on different or larger workloads; or attempt to translate from a publicly available trace to their system, risking mistranslation that leads to costly over-provisioning [91, 113].

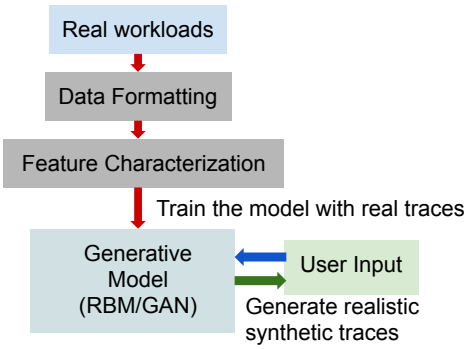


Figure 1: *System Overview.*

This project proposes to meet these challenges by building a suite of synthetic trace generators using *generative machine learning models* to accurately model large and complex real workload traces, laying the groundwork for improved trend prediction across the system stack. Unlike current synthetic models, generative machine learning models do not require manual parameterization of workload characteristics, and can operate at different levels of precision simultaneously. As a result, they can capture long-term, temporally correlated events that expert-designed synthetic systems cannot. Having realistic synthetic traces will provide an avenue for testing performance of simulations for new and upcoming systems at the research stage [114], and also help companies test their systems on the consumer's workload before selling the system to them. To realize the above goal, this project is broken into three mutually complementing subgoals: 1) **Generate synthetic workloads using generative models**, 2) **Develop testing framework to judge representativity of generated traces**, and finally 3) **Expand generative models to emulate variability and anomalies in real systems**.

Modelling Out of Distribution (OoD) values representing disk I/O and memory accesses is vital for performance evaluation of storage systems. These anomalies in real storage workloads cause sudden

increases in demand for resources, service disruptions, decrease in quality of service, and complicates autonomic resource management of data centers. Generating synthetic workloads that characterize and emulate these anomalies will help businesses with pre-planning their provisioning and resource utilization. The new knowledge generated by this project will be critical in predicting the performance of future hybrid systems, making it easy to test these systems with a single mixed load trace, as opposed to multiple traces for different aspects of the system. This will catalyze the development of newer prototypes and significantly reduce the total cost of ownership in storage systems. The yearly raw traces stored for a single HPc experiment can surpass 3 PB [29], with the added cost of storing the replicated data for high reliability and availability. On current drives with the lowest cost/bit, with mirroring, this leads to an **annual** overhead of over $100,000 per experiment [82, 33], for a total of millions saved over all scientific workloads.

B Generative Models for Synthesizing Storage Workloads

Using models and algorithms to analyze and understand the tremendous amount of information generated by systems can often be tricky. Generative models are a promising approach towards attaining this goal. Neural networks used as generative models have a number of parameters significantly smaller than the amount of data that can be used to train them with. Therefore, the models are forced to discover and internalize the essence of the data in order to generate it. Generative models aim at learning the true data distribution of the training set so that it can generate new data points with some variations. Discriminative models learn a statistical model to estimate the posterior probability $p(y|x)$, which is, how much a model believes that there is a feature x given an input workload y, compared to all possibilities it knows about. Generative models, on the other hand, try to learn a related function called joint probability $p(y,x)$, which is, how likely there is a workload y and a feature x at the same time. The fine-granularity provided by generative models helps link components of a distribution to create a completely new distribution, making it one of the top choices for generating realistic synthetic traces. The three most commonly used generative models for synthetic data generation are – Variational Autoencoders (VAE), Restricted Boltzmann Machines (RBMs), and Generative Adversarial Networks (GANs). Since autoencoders are usually used to convert generative models to discriminative models [92, 64], for this proposed work, the PIs focus on investigating the workload generative capabilities of RBMs and GANs.

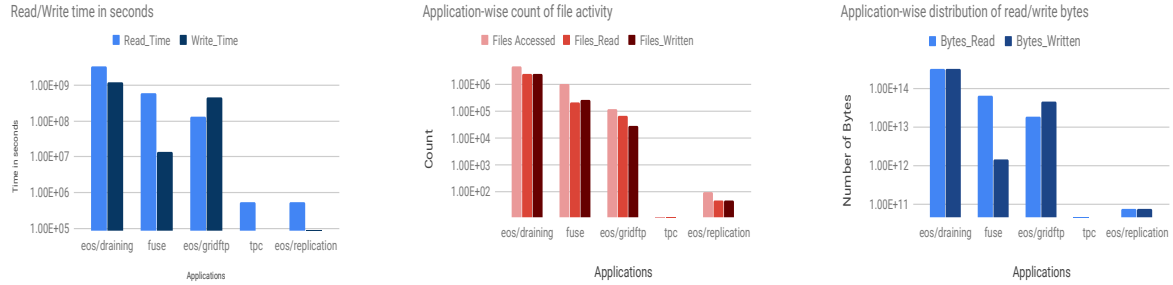


Figure 2: *The EOS logs have 8 applications for which they collect data. Out of those only 5 applications contain a representative set of features. We show the application-wise distribution of I/O, file activity and read/write times for the dataset.*

Dataset: The PIs used traces from the CERN EOS logs to train their preliminary generative model prototypes [95]. The EOS log files have a total of 57 fields to record file creation, I/O and

updates, though under GDPR restrictions only 30 were made available for analysis. As we can see in Figure 2, this dataset is highly imbalanced in terms of I/O distribution and number of file accesses, for the five experiments that have been considered. Since the results for the `eos/draining`, `fuse` and `eos/gridftp` are somewhat balanced, those are the experiments that are used as the target workload classes for the generative models. The EOS log keeps track of time taken to complete actions using open and closed file timestamps and read/write times. To identify a potential bottleneck on the system, it keeps track of the number of read/write calls. It also tracks changes in file sizes using bytes read from/written to files. The PIs will extend the prelimiary model train on CERN EOS to other enterprise traces from IBM Research, SNIA [94], NCAR [81], NetApp [21], and other open source traces like Backblaze [1].

B.1 Restricted Boltzmann Machines

Restricted Boltzmann Machines (RBM) are a flexible, generative model that can be used for dimensionality reduction, classification, regression, feature learning, and data modelling [45]. Invented by Geoffrey Hinton in 2006 [96], RBMs are shallow, two-layered neural networks consisting of a visible input layer and a hidden layer. These layers are connected by a fully bipartite graph, which means that all visible layer nodes are connected to all hidden layer nodes, but no two nodes in the same group are connected to each other. There are two additional layers of bias (hidden bias and visible bias), which makes RBMs different from other generative models. The hidden bias produces the activation on the forward pass during generation: $h^1 = S(v^{(0)T}W + a)$, where h^1 and v^0 are the corresponding vectors for the hidden and visible layers, S is a sigmoid activation function, and a is the hidden layer bias. The visible bias (v^1) is a second set of biases that the RBM uses during a backward pass for reconstruction of the input following $v^1 = S(h^{(1)}W^T + b)$, where b is the visible layer bias. The reconstructed input is always different from the actual input as there are no connections among the visible units and, therefore, no way of transferring information among themselves.

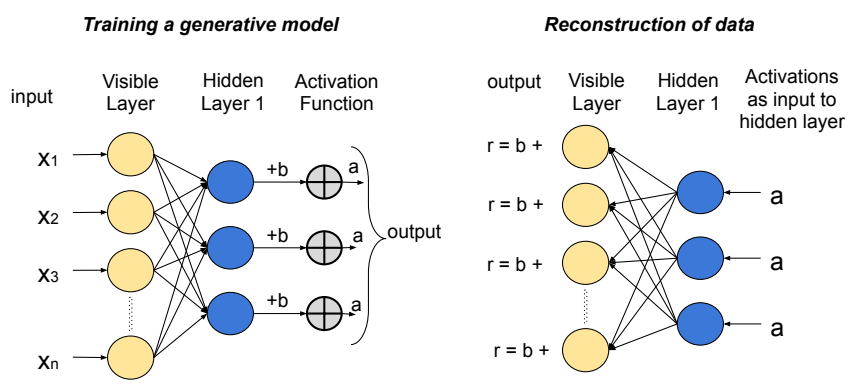


Figure 3: *Our RBM contains a single visible layer (input) with* 30 *nodes, one for each feature representing our workloads and a single hidden layer.*

B.1.1 Initial Results: Generating synthetic data points using RBMs

As a preliminary model, the PIs implemented an RBM that was trained on the aforementioned CERN EOS logs using enumerated application types ($1-3$) as target values. We tuned values like learning

rate and number of hidden layers for our RBM to study their impact on the reconstruction loss (squared error) and training time of the RBM.

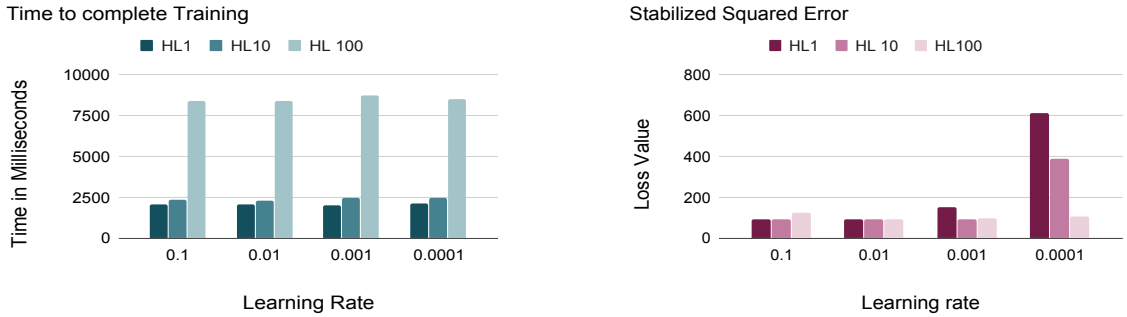
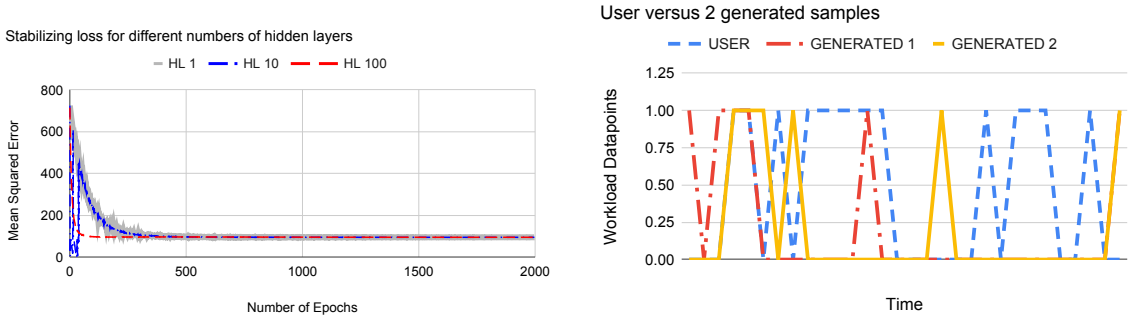


Figure 4: *Shows the variation in RBM training time and reconstruction loss with an increase in number of hidden layers and four different values of learning rate.*

The squared error or reconstruction loss is calculated as the difference between the neural networks guesses and the correct labels contained in the test set. The error is then backpropagated against the RBM's weights repeatedly in an iterative learning process until an error minimum is reached. As seen in part *a* of Figure 4, the learning rate does not have a large impact on the training time. Increase in the number of hidden layers from 1 to 100 increases the training time by almost 78%. Part *b* in Figure 4 shows the average stabilized mean squared error, which increases to over 84% for decreasing values of learning rate from 0.1 to 0.0001. Looking at the results from chart *b*, the preliminary RBM was trained used a learning rate 0.001.

Figure 5a shows the RBM's loss for different values of number of hidden layers, for increasing number of epochs. The unit for the squared error value is the same as the units of the target values, since our target values here are unitless, the squared error is also unitless. Despite the stabilized squared error value for every hidden layer being approximately 94, the number of epochs at which the model converged are different for each of the models. The model underfits for number of hidden layers 1, and overfits for number of hidden layers 100, with a gradual loss stabilizing for number of hidden layers 10. Therefore, the expected number of hidden layers would be closer to 10 than 100. The PIs will vary values from 20 to 50 to trace loss of the model. Figure 5b shows two generated



(a) *RBM reconstruction loss.* ***(b)*** *RBM Synthetic Data*

Figure 5: *Reconstruction loss over time for RBMs and RBM-generated [0,1] traces versus the user-supplied "ground truth."*

sample of datapoints, when a user provides the RBM with a sample set of datapoints. In this case, since we have 30 features representing each workload, the user provides a samples set of 30 feature values to the RBM. The RBM that has already been trained on different values for each feature, generates a set of 30 datapoints, every time a request is made for a new sample.

B.1.2 Proposed next steps: RBMs

The weights of the RBM are randomly initialized, therefore the difference between the reconstructions and the original input is often large. An interesting observation was made while attempting to reduce the reconstruction error value from 94. Reducing the number of features reduced the loss value. For certain subsets of features, especially the ones representing forward and backward seeks, the loss value went as low as 28. We suspect this happens due to stronger covariance between certain features in the dataset. From the preliminary results generated by this RBM prototype, we derive that this particular prototype of RBM works well for generating target values with a small subset of features, but the squared error reconstruction loss changes drastically with a higher number of features and with feature sets with low covariance between features. As next steps, the PIs plan to further investigate correlation between features to identify types of features and their effect of loss. They also plan to investigate other RBMs architectures like conditional RBMs and stacked temporal RBMs that are more suited for time series generation and prediction.

B.2 Generative Adversarial Networks

Generative adversarial networks (GANs) have a simple pretense: if you train a model on all available training data, and train a second model to try to come up with examples that the first model misclassifies, the second model will eventually converge to produce synthetic data indistinguishable from the data the first model was trained with [47]. For GANs, the models are artificial neural networks, known as the *discriminator* and the *generator*, and they are placed at odds as described in Figure 6. GANs have been extensively studied for synthetic image generation [16, 57, 109, 83].

Training a single neural network can be difficult due to the incredible number of choices involved: architecture, activation functions, optimization method, learning rate, and dropout rate, among others. GANs double all of those choices and add new complexities. Both the generator and the discriminator may forget samples they used earlier in their training, which can lead to the two networks getting caught in a stable cycle of solutions that do not improve over time [116]. One network may overpower the other network, such that neither can learn anymore [88]. The generator may not explore much of the possible solution space, only enough of it to find realistic solutions [57]. This last situation is known as *mode collapse*. Mode collapse is when the generator only learns a small subset of the possible realistic modes. For instance, if the task is to generate images of a house, the generator could learn to create only images of huts. The generator would have missed all of the other modes consisting of house of other sizes or shapes. The PIs believe they can get past these challenges by providing the GAN with a steady flow of novel

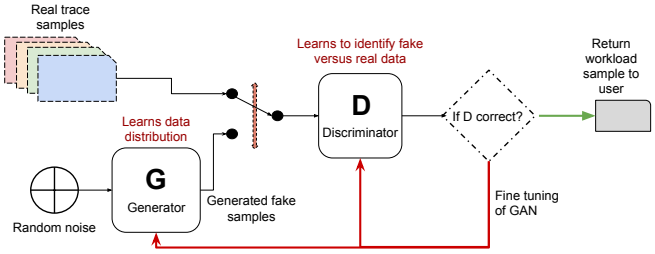


Figure 6: *GANs consist of two neural networks- the discriminator and the generator, which are in an adversarial feedback loop with each other.*

enterprise traces. This will allow the GAN's discriminator to update itself by learning from the characteristics of the new workloads. Batch normalization and mini-batch discrimination can also be used to handle model divergence and modal collapse [6].

B.2.1 Initial Results: Evolution of our GAN prototype

To check the viability of GANs for workload generation, the PIs first built a discriminative binary classifier model to classify normalized disk read and write speed, derived from the CERN logs, into two classes: high (> 1) bytes per second and low (< 1) bytes per second. The classifier had a suspiciously high validation accuracy of 96.7%, which is possible because this preliminary model only uses 4, hand-selected features.

The next step was to combine the discriminative model with a generative model to create a toy GAN. Both models consisted of four dense layers with alternating LeakyRELU activation layers, which accelerated convergence of the stochastic gradient descent, in turn reducing the computational burden on the model. The CERN data is in the form of a time series, which was cleaned and normalized as part of input pre-processing for the models. The total loss ($PredictionError_{discriminator} + InformationLoss_{generator}$) started at approximately 25 and kept rising for an increase in number of epochs. Again like with the RBMs, since our target value is unitless, the mean squared error value is also unitless. The intuition was that the models diverged as a dense neural network does not provide a well enough representation of time series data, as a feedback to the GAN's generator. This is because the layers in the a dense network are not connected to understand the correlation between past, present and future events.

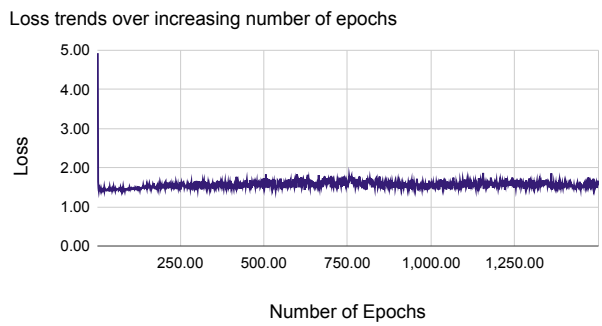


Figure 7: *GAN training loss* ($(loss_{disc.} + loss_{gen.})$).

The PIs then modified an implementation of semi-supervised InfoGAN [105] for the CERN timeseries data to visualize the trends in training losses with increasing epochs as shown in Figure 7. The training loss for the generator drops from approximately 5% to 1.4% in just 10 epochs, after which it stabilizes at a value of approximately 1.4% . Here, the training loss was calculated as the percentage of information loss in the generated workload.

B.2.2 Proposed next steps: GANs

To generate a statistically valid synthetic workload of high quality, the discriminator and the generator will need to be constantly updated with fresh data. If the generator does not update its weights based on inputs from the discriminator, it will not be able to generate samples that the discriminator can classify as real, and the models will never converge. If the discriminator is not updated frequently, non-representative workloads will be classified as real, reducing overall model fidelity. This generator will model workloads with similar functionality to the TPC-CH benchmarking tool mentioned by Funke [34] that produces a mixed workload.

Based on the preliminary results generated from the prototypes, the PIs propose the need to do an evaluation of the most suitable neural network architectures and generative models for time series generation and prediction. The PIs will first look at extending the discriminator binary classifier

model to include all of the existing CERN features, and eventually extend it to classify workloads in terms of the 5 CERN experiments available. The PIs also propose to evaluate different combinations of generators and discriminators with neural network architectures ranging from vanilla feed-forward neural networks (FFNN) to Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs) and Long/short-term memory networks (LSTMs). CNNs could be useful when working with a large number of features since the output parameters are defined on the convolution rather than on a per-unit basis, significantly restricting the number of features that need to be selected. Apart from each layer being defined as an output of previous layers in case of FFNNs, RNNs additionally have a link from the current layer to previous layers, which allows them to model present dependence on past events. LSTMs on the other hand, have no form of activation performed, allowing the same signal to flow back into the network for a longer period of time as compared to RNNs. This helps LSTMs represent time series better since the interaction between past and present events is sensitive to both very distant and recent events.

While surveying different applications of GANs, the PIs also realised that there are no readily available design guidelines for GANs. Every GAN is modified to fit a specific problem of either imputation, classification, prediction or generation. The PIs propose to come up with a general style guide on designing GANs for systems applications. Finally it will also be interesting to look at which of the above mentioned architectures would have the best performance accuracy for different kinds of datasets. Considering time series data, cross sectional data and pooling data (combination of time series and cross sectional data), the models that would most efficiently capture patterns of these should vary.

C Validating Generated Storage Workloads

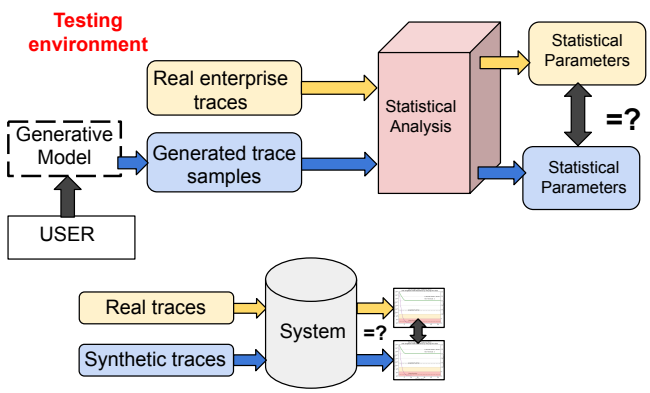


Figure 8: *This project will be validated by performing statistical similarity analysis between the real and generated traces, and by testing both workload and sample on a host system and against existing generators.*

The PIs propose to test the representativeness of generated workloads in two ways: testing on external systems and testing against existing generators. Additionally, they propose to develop a new heuristic of statistical accuracy for generated models to fill a gap in metrics to judge the representativity of synthetic I/O workloads. Figure 8 outlines the overall testing framework we propose to build.

C.1 Developing Machine learning based Statistical module

Existing generative models for image synthesis use metrics like inception score [103], Minkowski metric [98], visual sense [44], or existing GUIs like the Amazon Mechanical Turk to check quality of generated images. Measuring similarity between the generated and original time series is a related but distinct question; an exact match is less valuable than a looser but still representative generated trace. The most common existing vector similarity methods include using Euclidean distance [5], cosine similarity [14] and maximum mean discrepancy (MMD) [125].

Euclidean distance is mainly used to test numerical difference between two vector points by calculating the distance between them and does not lay emphasis on the difference in the data distribution. Cosine similarity measures the cosine value of the angle between two vectors, focusing more on the direction of the vector, rather than the size of the vector. MMD has been used to measure similarity between two distributions, but MMD does not account for density in distributions. Since this project seeks to detect and replicate temporal anomalies (Section D), there needs to be a provision to check for changes in density of the workload.

To address these inadequacies, the PIs propose to develop a novel machine learning based statistical model to identify similarities between two distributions. Each distribution is characterized by a number of features. Statistical correlations between distributions will be identified using manually or automatically selected metrics such as frequency of events, time of occurrence (start/end time) of events, or range of values for events. The PIs will also implement techniques like distributed clustering to verify the similarity model (since it will almost certainly be too slow to run inline). We will consider our project successful if more than 80% of the datapoints from the generated workload are a part of the same distribution cluster as that of the datapoints from the corresponding real workload.

C.2 Workload Testing

After statistical verification of the generated workloads, the PIs will test the workload samples on external systems and also compare quality of generated samples with traces generated by existing synthetic workload generators like Filebench, Impressions and VdBench.

C.2.1 Testing against external systems

The PIs propose to compare the values of performance metrics of the generated workload samples with its corresponding real traces on a host system. Performance metrics that will likely be calculated are transaction time, arrival rate of requests, average request response time, error rate, CPU usage, system reliability and availability. We will consider our project successful if the system provides the same values for the performance metrics, when tested with both the generated workload sample and the real workload sample. In the case of a hybrid workload AB this test would involve testing system AB with workload AB and independently with workloads A and B to see if they return similar performance numbers.

C.2.2 Testing against existing generators

Another way to test the workloads generated by this project would be to test them against samples generated by existing workload generators. For example, Filebench promises generated I/O traces that accurately reconstruct real application I/O mix, but fail to effectively capture I/O dependencies. There are two comparisons that can be made of the traces generated by this project and the traces generated by Filebench. Performing statistical analysis using machine learning techniques like clustering will provide us information about how close the generated traces are to the Filebench traces. Testing the

performance of a host file system like IBM's GPFS or Windows NTFS with our generated traces and Filebench traces will provide us with performance metrics for both. We will consider our project successful if the performance numbers generated by our traces is similar or better than the numbers generated by the Filebench traces.

D Capturing and Emulating Workload Anomalies

To realistically emulate a time series with higher order effects, a model must include variation with different periods. Over shorter trace segments, this long-term variability looks like an outlier. The PIs propose to identify outliers and anomalies in the training sets and reproduce these patterns in generated workloads. Detecting anomalies in time series is a well researched area with many possible options for this project to use, such as signal analysis or regression [10, 18, 123]. Another way to detect inconsistencies is by combining an outlier density evaluation model with a predictive uncertainty estimation on the density model [63]. The PIs intend to explore an assortment of these techniques as this work matures as well as methods that use generative models for anomaly detection [104, 22].

While the PIs have a stretch goal of detecting and replicating a wide swath of anomalies, for this proposal the PIs will focus on temporal anomalies such as burstiness. Burstiness is defined as inconsistencies in workload trends, often related to atypical request types, arrival times, or overall load volume [110]. In real workloads, temporal outliers can cause sudden increases in demand for resources and an overall decrease in the quality of services [107]. For example, in Figure 9 we see two clear read and write bursts as well as several periods of high write intensity over a day of data collection for a scientific workload. While we expect that our generated workloads will be able to emulate the writes, we need to be able to occasionally detect read/write bursts in generated workloads for them to be look authentic and not languish in the "uncanny valley" of storage traces.

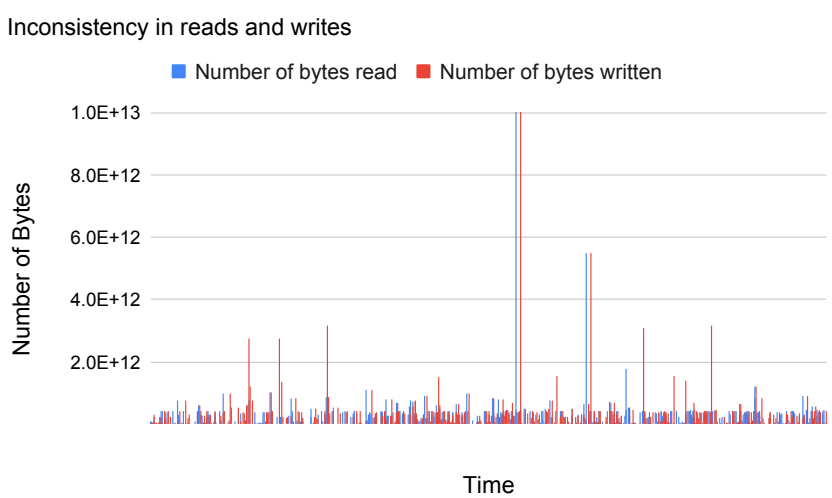


Figure 9: *A typical day of CERN EOS file I/O logs for the CMS experiment has aperiodic spikes in activity.*

For this project the PIs will characterize burstiness in multiple real workloads by studying trace access patterns, looking over the event timestamps to find out irregularities in distribution of reads and writes, and studying unequal file accesses by looking over file IDs and their open and close metadata. Part of the analysis on real enterprise workloads will include evaluating existing state of the art quantitative measures like gini index and entropy, used to quantify burstiness. Following a

study of anomaly characterization in real workloads, the PIs draw inspiration from existing anomaly generation techniques [65, 97, 40], to compare four different methods for injecting anomalies into our generated workload samples:

1. **Transformative Metrics:** Generative models can train using normal data and determine metrics like magnitude and duration to identify transformations from normal to outlier data. Once these metrics are determined, they can be supplied to the model as parameters to create new outlier distributions.
2. **Generative Adversarial Networks:** A GAN generator can be trained on anomalous data to generate outlier distributions. These outlier distributions can then be provided as feedback to the discriminator, that can use this data as additional training information.
3. **Regulators:** Training a generative model on a normal distribution and using a regulator combined with a random timestep generator to insert random outliers within the generated normal distribution.
4. **Hybrid Workloads:** We can also assign a unique workload class for anomalous distributions, and generate a hybrid workloads, a combination of normal and anomalous distributions.

The PIs will initially use GANs to generate burstiness. There are several challenges associated with outlier generation using GANs: **(1) GANs trained on outlier data often suffer from modal collapse.** This is due to the unbalanced number of outlier modes in time series datasets. We will tackle this by first training our generator on outliers to produce additional outlier datapoints, which will be added to our discriminator training set to make the dataset more balanced. **(2) Infrequency of outliers in datasets provides insufficient information for training machine learning models, causing overfitting.** Specifically collecting datasets with higher distributions of outliers to train our discriminator will help our GAN generate more outlier datapoints. **(3) It is challenging to estimate and emulate the exact random timesteps at which outliers will be generated.** This can be fixed by adding a random timestep generator to the generative model.

E Related Work

E.1 Synthetic Workload Generators:

One of the earliest synthetic workload generators was introduced by Robert Bodnarchuk in 1991 [15]. This generator was meant to be used in experimental studies for the UNIX/NFS distributed file system and the goal was to achieve an accurate, flexible and reproducible workload. Although this work did not perform any statistical analysis of their generated samples, Bodnarchuk provided experimental evidence that synthetic workloads could emulate characteristics of real workloads. Multiple subsequent papers discussing synthetic workload generators [30, 58, 60, 26, 59] highlighted virtual level implementations of synthetic workload generators, but did not store load patterns for future regeneration [76, 56, 39]. Our proposal draws inspiration from the historical progress made in the synthetic trace generation field.

Automating the construction of representative synthetic workloads poses significant challenges. Synthetic trace generators like GISMO [51],ProWGen [17], WebTraff [74] and MediSyn [118] were novel, but failed to capture a representative performance analysis of the systems. Generators like NectGen [106, 25], Tmix [117] and DynBench [12], generated synthetic workloads without *a priori* knowledge about the represented applications they are representing, which meant that the model could miss certain parameters of the application [46, 2].

Table 1: *Synthetic workload generator properties [8, 122, 4, 7]*

Tool	Open Source	Adaptable Features	Scalable	User Flexibility
Bonnie	✓	✗	✗	✗
I/Ometer	✓	✗	✗	✗
aio-stress	✓	✓	✗	✗
Buttress	✓	✗	✓	✗
Fstress	✓	✗	✓	✗
Fio	✓	✗	✗	✗
Auto-pilot	✓	✗	✗	✗
IOzone	✓	✗	✓	✗
SPECsfs	✓	✗	✓	✗
Filebench	✓	✗	✗	✓
Impressions	✓	✗	✓	✓
FFSB	✗	✗	✓	✗
DBench	✓	✓	✗	✗
FsBench	✓	✓	✗	✗
VdBench	✓	✗	✗	✗
This Project	✓	✓	✓	✓

More recent synthetic workload generators include GT-CWSL [9] and Mimesis [23]. GT-CWSL generates the specifications that are provided as input to a synthetic workload generator, but fails to cover important workload attributes like temporal locality, file size, request size, or file popularity. The traces generated by Mimesis mimic the original workload, but the accuracy of the simulation can only be verified by benchmarking an existing system [23]. Properties of some additional existing workload generators are summarized in Table 2. Using generative models allows us to continuously update our synthetic traces with new, hybrid and improved samples derived from existing real workloads, with minimal human intervention. To summarize, the PIs propose to build a new trace generation suite because existing tools creating synthetic workloads: **(1)** cover a limited number of externally selected trace characteristics, **(2)** lack quantifiable statistical similarity to the characteristics of the real traces, and **(3)** can only be used to benchmark existing systems versus predicting future system performance.

E.2 Generative Modeling for Time Series:

Variational Autoencoders (VAE) have been applied to time series datasets for anomaly detection [126], reliable confidence estimation of missing values [35], learning graph embedded tasks [102] and generating time series data at scale [65]. Most of these previous VAE models have been replaced by the use of GANs simply due to the added level of flexibility being offered by the double feedback between the generator and the discriminator of the GAN [31]. Other generative models that have been used to analyze and generate time series datasets are Deep Belief Networks [48], probabilistic graphical models [61], RBMs have been used in the form of conditional RBMs for reconstruction of non-linear time series data [112], symbolic time series analysis [99], stacked RBMs for time series prediction [50] and other static applications as mentioned in [111].

In the last five years GANs have been used for different applications [83, 68, 100, 79, 66, 38, 28, 54, 124, 109, 49, 27, 86], all of which where either related to image/video synthesis or emulating human behaviour. Starting 2019, there have been attempts at trying to generate and analyze time series data using LSTMs, Wasserstein GANs, conditional GANs and other generative models. Most of these applications were to generate trends in financial markets [108, 62], emulate ECG and PPG [24, 16], rhythm generation [127] and anomaly detection [69]. To the best of our knowledge, GANs have yet

to be used to generate system workloads, making this project one of its kind. In comparison to these prior works, this project's goal is to use GANs to build a system which can characterize those features of a realistic workload, which cannot be derived through manual/human analysis. The PIs propose to setup RBM and GAN-based time series workload generators as a starting point. In the future they plan to extend their research to exploration of other generative models that can be used to generate and predict time series sequences.

F Intellectual Merit

This research will be the first to introduce a suite of *self-improving synthetic workload generative models*, which can generate an unlimited amount of sample workloads, given a set of system characteristics, with minimum human effort. Apart from creating workloads that emulate a large set of characteristics of their corresponding real traces, the models will be tuned to create scalable workloads with hybrid characteristics. In the case of hybrid systems, such as multiple databases, this project will significantly reduce the time to instrument workload analysis by allowing new algorithms to run against a single generated workload instead of multiple workloads per characteristic and sub-system. The PIs analysis of different generative models for different types of traces will contribute to the understanding of how inputs and parameters like learning rate and number of hidden layers affect the outputs of a generative model. The expansion of the generative models to include capturing and emulation of anomalies in real workloads will allow businesses to anticipate spikes in system usage and improve resource provisioning.

G Broader Impacts

This project will produce customizable realistic workloads to predict the performance of future hybrid systems, which will catalyze the development of newer prototypes and significantly reduce the total cost of ownership in storage systems. Apart from having a significant impact on the systems groups such as databases, cloud and distributed systems, this project also has the potential to be applied to the gaming community. Generative models can be used to create realistic simulations of gamer environment, that can be used to test games before releasing them to the public. This research on scalable time series generation can also be readily translated to other areas with trace availability issues like financial stock market and economic consumption, to improve resource and production management. The traces generated by this project will pioneer future students and researchers creating and benchmarking new and innovative system designs. This set of synthetic workload generation models would serve as an excellent foundation to build and predict new storage, databases and distributed systems.

G.1 Data Privacy

This project's idea to generate *real-like* traces that do not contain real user or application information supports anonymizing sensitive user and corporate data. In light of GDPR, there is currently an underlying tension between privacy and exchanging high quality open source data [19, 77]. Companies are looking for ways to obtain, process and utilize personal data without violating the new rules. Existing data anonymization techniques like randomization and generalization often fail to successfully hide sensitive user and application data, while simultaneously capturing trends provided by such data [101]. With the increasing gap between the amounts of captured versus sharable information, it is necessary to find a way to replicate real user data. Unlike hand engineered synthetic data which is simplistic and biased [43], machine learning based synthetic generations are realistic and representative.

While the generative models in Section B are trained on real user or application data, the resulting synthetic traces are free of any directly identifying information. This allows system maintainers to share their I/O trend data with contractors or clients without further compromising user privacy. This makes it easier for businesses to perform trend analysis, system performance testing, and validation before shipping storage systems to consumers. Highly representative synthetic workloads also facilitate better collaboration between system vendors and consumers by allowing privacy-preserving trend sharing. Finally, the PIs intend to present a version of this project to colleagues at Emory Medical School to explore the overlaps between I/O trend analysis and a variety of medical time series analyses, potentially avoiding the overhead of medical privacy protection (HIPPA).

G.2 Educational Plan

Curriculum Development Activities UC Santa Cruz (UCSC) encourages curriculum changes that have a meaningful impact on the educational community. Our proximity to the Silicon Valley reflects directly through the strong influence of the technical workforce on the curriculum development at UCSC. This relation between the Silicon Valley workforce and UCSC is further strengthened by the constant flow of outgoing computer science undergraduates that enter the workforce and, incoming skilled industry fellows that re-join the university to obtain their Master's or PhD degrees. PI Long has been teaching courses like Computer Systems and C Programming (undergraduate-level), Storage Systems (graduate-level), Advanced Computer Security (graduate-level) and Introduction to Operating Systems (undergraduate-level) over the years and is planning to integrate the benchmarking tool generated through this research project into the curriculum and encourage students to test their systems project using this tool. Focusing on developing tools that support upcoming systems research and education is essential in a region where a large portion of our nation's systems are created, developed, and maintained.

PI Wildani will take the output from this work and develop an applications of GAN module for her existing neural computation course that has enrollment from students across disciplines, with a goal of bringing more students from non-traditional backgrounds into systems research. She will also be integrating a unit on GANs in her current graduate systems course. These modules and units will be made publicly available to help accelerate training in machine learning for systems.

Research Development and Broadening Participation Wildani is building an interdisciplinary lab at Emory centered around systems analysis and quantification. This project will help attract students focused on cutting-edge machine learning projects to her SymBioSys group at Emory University. Long was one of the founder's of the Storage Systems Research Center(SSRC) at UCSC, that has been producing multiple computer scientists who mainly focus on coming up with innovative implementations of storage systems. The graduate students at SSRC and Emory are actively encouraged to mentor undergraduate students, and we plan to recruit some of them for this project. This hands-on work will provide these young minds with an in-depth knowledge of system characteristics and testing, and connect them with our industry partners. This project will provide support to two PhD candidates, both specializing in applying machine learning to systems.

Both PIs are committed to improving representation of women and underrepresented minoritied (URMs) in computing. Wildani is the founder of Project Hypatia (project-hypatia.org), a forum designed to publicize fundamental CS research led by women. She has also been a panelist for computational ethics and privacy. The PIs have budgeted funds for cross campus personnel exchange to expose students to institutions with greater proportions of African-American (Emory) and Hispanic

(UCSC) students. The PIs also have planned a series of lectures at HBCUs and HSI institutaions in their respective locales to: 1) introduce URMs to graduate level systems research, 2) showcase questions of ethics and bias in ML, and 3) mentor students who are interested in graduate study in systems. (See: BPC Plan)

H Timeline and Evaluation Plan

The three-year research plan for this project will leverage the complementary strengths of Emory and UCSC. The project will be implemented in three segments to achieve the research objectives outlined in the project summary: 1) **Using generative models to generate synthetic workloads**, 2) **Developing test framework to judge representativity of generated traces**, and 3) **Expanding generative models to capture and emulate anomalies in real systems**.

Table 2: *Project Timeline*

Task	Year 1	Year 2	Year 3
Trace collection and formatting	✓	✓	✓
Building and training prototypes	✓		
Validating Models		✓	✓
Statistical Analysis		✓	✓
Fine tuning the model for scalability		✓	
Uncertainty Estimation			✓
Open source publications	✓	✓	✓

H.1 Evaluating Intellectual Contributions

Once trained, the generative model, when combined with a discriminator model is expected to predict workload classes when provided with a set of parameters. This will be followed by the development of prototype synthetic workloads that can be used as baseline to improve the discriminative and the generative model. Prototype workloads will be evaluated on existing systems like GPFS and compared against synthetic workloads generated by other workload generators like Filebench and I/Ometer. Our project will be considered successful if the performance numbers of the host system are the same for both the sample and pre-existing workload generated by Filebench. In order to leverage the expertise of the ML community and maintain a pipeline for interested undergraduates, we will run the project as an open source project.

H.2 Evaluating Broader Impacts

We will host our generative model prototypes on open source platforms and share outputs. Once the prototype starts generating system I/O traces successfully, we will train our models on real workloads from other fields like finance, medicine, and seismology to see what kind of data we can predict and generate. We will leverage Emory's collaboration with Emory medical school to test our model's capability to generate traces from the medical community. Our modelled workloads will be considered successful under two conditions: 1) **They manage to generate real-like user test cases that can be used to check performance of systems without exposing sensitive information about the real users** and 2) **They system returns similar performance numbers with both the generated and the real workloads**.

I Results From Prior NSF Support

"CRII: CSR: Skeletor: Building a Platform for Quantitative Workload Characterization," Wildani, NSF Award Number CNS–1755958, $174,997, 9/1/18–9/1/20

Intellectual Merit: This ongoing project builds a method to select predictive, high-information metrics to describe modern workloads coupled with a way of isolating workloads within a system to open a new area of inquiry for storage system tuning and design. This re-opens the field of workload characterization and helps the systems community handle the massive, decentralized data on the horizon from consumer systems such as self-driving cars or augmented reality [20].

Broader Impacts: This project gives the systems community a standard set of metrics to express details about workloads used in papers where the data is proprietary, improving the uptake and reproducibility of systems research.

"Automatic Storage and Network Contention Management for Large-Scale High-Performance Computer Systems," Long, Miller, NSF award number CNS–1528179, $500,000, 9/1/15–8/31/18

Intellectual Merit: ASCAR, an Automatic Storage Contention Alleviation and Reduction system, uses machine learning methods combined with several heuristics to discover the fittest control strategy for the specific workload running on the specific system [73, 71, 70]. First of its kind to determine control strategy for system specific workloads without requiring prior knowledge of the hardware or software. Existing contention management methods rely on mathematical modeling of the system, which is becoming increasing hard and inaccurate as the system scales out.

Broader Impact: Has the potential to give US based systems an automated way to resolve issues and allows developers to dedicate more time and resources towards other problems.

"CSR: Small: A Multi-Layered Deniable Steganographic File System", Long, Schwarz, NSF Award Number CNS-1814347, $500,000, 9/1/17–9/30/19

Intellectual Merit: First to implement a steganographic file system whose mere existence can credibly be denied [11].. Previous attempts have not successfully been deniable and are detectable by traditional forensic methods. The system uniquely rebuilds itself even when under direct attack.

Broader Impacts: Designed for journalists, human rights advocates, and undercover law enforcement, Artifice is used where data privacy and deniability is paramount.

"I/UCRC: A Single-Site I/UCRC Center for Research in Storage Systems (CRSS)", Miller, Long, NSF award number IIP–1266400, $445,000, 7/1/13–6/30/18 The Center for Research in Storage Systems (CRSS) is a collaboration between academia and industry to explore and develop new technologies and techniques to improve the manageability, scalability, security, reliability, longevity, and performance of storage systems. CRSS researchers have won many best paper awards for system performance optimizations. Other publications: [119, 3, 72, 55, 87, 78, 121, 42, 84, 85, 37, 89, 93, 52, 120, 41, 80, 90, 53, 115].

Intellectual Merit: CRSS researchers conduct high-quality, industrially-relevant fundamental research, often with regular feedback from CRSS industrial members. CRSS has conducted cutting-edge research on archival storage, secure storage, data deduplication, non-volatile memories, performance tuning, benchmarking and shingled disk.

Broader Impact: CRSS faculty and students have given more than two dozen talks at universities and industry around the world, disseminating our research. CRSS has produced more than 10 Ph. D. graduates (including more than five women) and over a dozen M. S. graduates. CRSS also involves undergraduates in our research, including several who have gone on to graduate study.

References Cited

[1] Hard drive data and stats by backblaze. `https://www.backblaze.com/b2/hard-drive-test-data.html`. Date Retrieved 08/27/2019.

[2] Abdolreza Abhari and Mojgan Soraya. Workload generation for YouTube. *Multimedia Tools and Applications*, 46(1):91, June 2009.

[3] Ian F. Adams, Mark W. Storer, Avani Wildani, Ethan L. Miller, and Brian A. Madden. Validating storage system instrumentation. In *Proceedings of the 21st International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS '13)*, August 2013.

[4] Nitin Agrawal. Generating realistic impressions for file-system benchmarking. In *7th USENIX Conference on File and Storage Technologies (FAST 09)*, San Francisco, CA, Feb. 2009. USENIX Association.

[5] Robert J Alcock, Yannis Manolopoulos, et al. Time-series similarity queries employing a feature-based approach. In *7th Hellenic conference on Informatics*, pages 27–29, 1999.

[6] Javad Amirian, Jean-Bernard Hayet, and Julien Pettré. Social ways: Learning multi-modal distributions of pedestrian trajectories with gans. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2019.

[7] Eric Anderson, Mahesh Kallahalla, Mustafa Uysal, and Ram Swaminathan. Buttress: A toolkit for flexible and high fidelity i/o benchmarking. In *Proceedings of the 3rd USENIX Conference on File and Storage Technologies*, pages 45–58. USENIX Association, 2004.

[8] Jens Axboe. Fio–flexible I/O tester synthetic benchmark. `https://www.storagereview.com/fio_flexible_i_o_tester_synthetic_benchmark`, 2005.

[9] Arshdeep Bahga and Vijay Krishna Madisetti. Synthetic workload generation for cloud computing applications. *Journal of Software Engineering and Applications*, 4(07):396, July 2011.

[10] Paul Barford, Jeffery Kline, David Plonka, and Amos Ron. A signal analysis of network traffic anomalies. In *Proceedings of the 2nd ACM SIGCOMM Workshop on Internet measurment*, pages 71–82. ACM, 2002.

[11] Austen Barker, Staunton Sample, Yash Gupta, Anastasia McTaggart, Ethan L Miller, and Darrell DE Long. Artifice: A deniable steganographic file system. In *9th {USENIX} Workshop on Free and Open Communications on the Internet ({FOCI} 19)*, 2019.

[12] Binoy Ravindran Charles Cavanaugh Bharath Yanamula Russ Brucks Behrooz Shirazi, Lonnie Welch and Eui nam Huh. DynBench: A dynamic benchmark suite for distributed real-time systems. In *IPPS/SPDP Workshops*, volume 1586, pages 1335–1349, Nov. 1999.

[13] Tan Wei Zhou MengChu Fan Yushun Zhang Jia Bi Jing, Yuan Haitao and Li Jianqiang. Application-aware dynamic fine-grained resource provisioning in a virtualized cloud data

center. *IEEE Transactions on Automation Science and Engineering*, 14(2):1172–1184, Apr. 2015.

[14] Filippo Maria Bianchi, Lorenzo Livi, Alberto Ferrante, Jelena Milosevic, and Miroslaw Malek. Time series kernel similarities for predicting paroxysmal atrial fibrillation from ecgs. In *2018 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2018.

[15] Robert Bodnarchuk and Richard Bunt. A synthetic workload model for a distributed system file server. In *Proceedings of the 1991 ACM SIGMETRICS conference on Measurement and modeling of computer systems*, pages 50–59. ACM, 1991.

[16] Eoin Brophy, Zhengwei Wang, and Tomas E Ward. Quick and easy time series generation with established image-based gans. *ArXiv*, abs/1902.05624, Feb. 2019.

[17] Mudashiru Busari and Carey Williamson. Prowgen: a synthetic workload generation tool for simulation evaluation of web proxy caches. *Computer Networks*, 38(6):779–794, Apr. 2002.

[18] Antonio Candelieri. Clustering and support vector regression for water demand forecasting and anomaly detection. *Water*, 9(3):224, 2017.

[19] Hsinchun Chen, Roger HL Chiang, and Veda C Storey. Business intelligence and analytics: From big data to big impact. *MIS quarterly*, 36(4):1165–1188, 2012.

[20] Si Chen and Avani Wildani. Chasing the signal: Statistically separating multi-tenant i/o workloads. *Machine Learning for Systems*, 2018.

[21] Tiberiu Chis and Peter G Harrison. iswom: The incremental storage workload model based on hidden markov models. In *International Conference on Analytical and Stochastic Modeling Techniques and Applications*, pages 127–141. Springer, 2013.

[22] Sophine Clachar. *Novelty detection and cluster analysis in time series data using variational autoencoder feature maps*. PhD thesis, Dept. of Comptuer Science, Univ. of North Dakota, GrandForks, ND, USA, 2016.

[23] Yi Lu Cristina L. Abad, Huong Luu and Roy H. Campbell. Metadata traces and workload models for evaluating big storage systems. In *2012 IEEE Fifth International Conference on Utility and Cloud Computing*, pages 125–132. IEEE, Nov 2012.

[24] Anne Marie Delaney, Eoin Brophy, and Tomas E Ward. Synthesis of realistic ecg using generative adversarial networks. *ArXiv*, abs/1909.09150, Sept. 2019.

[25] Jerome A. Rolia Diwakar Krishnamurthy and Shikharesh Majumdar. A synthetic workload generation technique for stress testing session-based systems. *IEEE Transactions on Software Engineering*, 32(11):868–882, Nov. 2006.

[26] Joel L. Wolf Dominique Thiebaut and Harold S. Stone. Synthetic traces for trace-driven simulation of cache memories. *IEEE Transactions on Computers*, 41(4):388–410, Apr. 1992.

[27] Chris Donahue, Julian McAuley, and Miller Puckette. Adversarial audio synthesis. *arXiv*, 1802.04208, Feb. 2018.

[28] Jeff Donahue, Philipp Krähenbühl, and Trevor Darrell. Adversarial feature learning. *ArXiv*, abs/1605.09782, May 2016.

[29] Dirk Duellman. Private Communication, 2019.

[30] Winfried Dulz and Stefan Hofmann. Grammar-based workload modeling of communication systems. In *Proceedings 5th International Conference on Modeling Techniques and Tools for Computer Performance Evaluation*, Feb. 1991.

[31] Cristóbal Esteban, Stephanie L. Hyland, and Gunnar Rätsch. Real-valued (medical) time series generation with recurrent conditional gans. *ArXiv*, abs/1706.02633, June 2018.

[32] Christian Ferdinand and Reinhard Wilhelm. Efficient and precise cache behavior prediction for real-time systems. *Real-Time Systems*, 17(2-3):131–181, Nov. 1999.

[33] Andy Ferris. Tape backup: How much is tape really costing your business? `https://www.igneous.io/blog/hidden-costs-tape-backup`, 2018. Date Retrieved 04/17/2018.

[34] Alfons Kemper Florian Funke and Thomas Neumann. Benchmarking hybrid OLTP and OLAP database systems. In *Proceedings of Datenbanksysteme für Business, Technology und Web (BTW)*, pages 390–409, 2011.

[35] Vincent Fortuin, Gunnar Rätsch, and Stephan Mandt. Multivariate time series imputation with variational autoencoders. *ArXiv*, abs/1907.04155, July 2019.

[36] Intel Developers Forum. I/ometer(workload generator). `https://gerardnico.com/io/iometer`, 1998.

[37] Joel Frank, Shayna Frank, Lincoln Thurlow, Thomas Kroeger, Ethan L. Miller, and Darrell D. E. Long. Percival: A searchable secret split datastore. In *Proceedings of the 31st IEEE Conference on Mass Storage Systems and Technologies*, June 2015.

[38] Yucheng Fu and Yang Liu. Bubgan: Bubble generative adversarial networks for synthesizing realistic bubbly flow images. *Chemical Engineering Science*, 204:35–47, Aug. 2019.

[39] Gregory R. Ganger. Generating representative synthetic workloads: An unsolved problem. In *Proceedings of the Computer Measurement Group (CMG) Conference*, pages 1263–1269, Dec. 1995.

[40] Justin Gottschlich. Paranom: A parallel anomaly dataset generator. *arXiv preprint arXiv:1801.03164*, 2018.

[41] Preeti Gupta, Darrell D. E. Long, Ethan L. Miller, David S. H. Rosenthal, and Avani Wildani. Effects of prolonged media usage and long-term planning on archival systems. In *Proceedings of the 32nd IEEE Conference on Mass Storage Systems and Technologies*, May 2016.

[42] Preeti Gupta, Avani Wildani, Daniel Rosenthal, Ethan L. Miller, Ian Adams, Christina Strong, and Andy Hospodor. An economic perspective of disk vs. flash media in archival storage. In *Proceedings of the 22nd International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS '14)*, September 2014.

[43] Ankur Handa, Viorica Patraucean, Vijay Badrinarayanan, Simon Stent, and Roberto Cipolla. Understanding real world indoor scenes with synthetic data. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4077–4085, June 2016.

[44] Jeffrey Heer and Ben Shneiderman. Interactive dynamics for visual analysis. *Queue*, 10(2):30, 2012.

[45] Geoffrey E Hinton. A practical guide to training restricted boltzmann machines. In *Neural networks: Tricks of the trade*, pages 599–619. Springer, 2012.

[46] Wagner M. Santos Hugo E. S. Galindo, Sergio M. L. Galdino, and Jose Paulo Pires. Synthetic workload generation for capacity planning of virtual server environments. In *2009 IEEE International Conference on Systems, Man and Cybernetics*, pages 2837–2842. IEEE, Oct. 2009.

[47] Mehdi Mirza Bing Xu David Warde-Farley Sherjil Ozair Aaron Courville Ian Goodfellow, Jean Pouget-Abadie and Yoshua Bengio. Generative adversarial nets. In Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27*, pages 2672–2680. Curran Associates, Inc., 2014.

[48] Takumi Ichimura and Shin Kamada. Adaptive learning method of recurrent temporal deep belief network to analyze time series data. In *2017 International Joint Conference on Neural Networks (IJCNN)*, pages 2346–2353. IEEE, May 2017.

[49] Yunseok Jang, Gunhee Kim, and Yale Song. Video prediction with appearance and motion conditions. *arXiv*, abs/1807.02635, July 2018.

[50] Peng Jiang, Cheng Chen, and Xiao Liu. Time series prediction for evolutions of complex systems: A deep learning approach. In *2016 IEEE International Conference on Control and Robotics Engineering (ICCRE)*, pages 1–6. IEEE, Apr. 2016.

[51] Shudong Jin and Azer Bestavros. Gismo: A generator of internet streaming media objects and workloads. *SIGMETRICS Perform. Eval. Rev.*, 29(3):2–10, Dec. 2001.

[52] Stephanie Jones, Ahmed Amer, Ethan L. Miller, Darrell D. E. Long, Rekha Pitchumani, and Christina Strong. Classifying data to reduce long-term data movement in shingled write disks. *ACM Transactions on Storage*, 12(1), February 2016.

[53] Stephanie N. Jones, Ahmed Amer, Ethan L. Miller, Darrell D. E. Long, Rekha Pitchumani, and Christina R. Strong. Classifying data to reduce long term data movement in shingled write disks. In *Proceedings of the 31st IEEE Conference on Mass Storage Systems and Technologies*, May 2015.

[54] Deepak Pathak Trevor Darrell Alexei A Efros Oliver Wang Jun-Yan Zhu, Richard Zhang and Eli Shechtman. Toward multimodal image-to-image translation. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pages 465–476. Curran Associates Inc., 2017.

[55] Yangwook Kang, Thomas Marlette, Ethan L. Miller, and Rekha Pitchumani. Muninn: a versioning key-value store using object-based storage model. In *Proceedings of the 7th Annual International Systems and Storage Conference (SYSTOR 2014)*, June 2014.

[56] Wei-Iun Kao and Ravishankar K. Iyer. A user-oriented synthetic workload generator. In *Proceedings of the 12th International Conference on Distributed Computing Systems*, pages 270–277. IEEE, June 1992.

[57] Salome Kazeminia, Christoph Baur, Arjan Kuijper, Bram van Ginneken, Nassir Navab, Shadi Albarqouni, and Anirban Mukhopadhyay. Gans for medical image analysis. *arXiv*, abs/1809.06222, Sept. 2018.

[58] Daniel Lee Kiskis. *Generation of Synthetic Workloads for Distributed Real-time Computing Systems*. PhD thesis, Ann Arbor, MI, USA, 1992. UMI Order No. GAX93-08362.

[59] Daniel Lee Kiskis and Kang G. Shin. Generating synthetic workloads for real-time systems. In *IFAC Proceedings Volumes*, pages 103–105. ScienceDirect, May 1991.

[60] Daniel Lee Kiskis and Kang G. Shin. SWSL: a synthetic workload specification language for real-time systems. *IEEE Transactions on Software Engineering*, 20(10):798–811, Oct. 1994.

[61] D. Koller and N. Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.

[62] Adriano Koshiyama, Nick Firoozye, and Philip Treleaven. Generative adversarial networks for financial trading strategies fine-tuning and combination. *ArXiv*, abs/1901.01751, Jan. 2019.

[63] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, pages 6402–6413, 2017.

[64] Alex Lamb, Vincent Dumoulin, and Aaron C. Courville. Discriminative regularization for generative models. *ArXiv*, abs/1602.03220, Feb. 2016.

[65] Nikolay Laptev. Anogen: Deep anomaly generator. Technical report, Technical Report. Facebook. https://research. fb. com/wp-content/uploads . . . , 2018.

[66] Minhyeok Lee and Junhee Seok. Controllable generative adversarial network. *IEEE Access*, 7:28158–28169, Feb. 2017.

[67] Kristen LeFevre, David J DeWitt, and Raghu Ramakrishnan. Workload-aware anonymization. In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 277–286. ACM, 2006.

[68] Aykut Erdem Levent Karacan, Zeynep Akata and Erkut Erdem. Learning to generate images of outdoor scenes from attributes and semantic layouts. *ArXiv*, abs/1612.00215, Dec. 2016.

[69] Dan Li, Dacheng Chen, Baihong Jin, Lei Shi, Jonathan Goh, and See-Kiong Ng. Mad-gan: Multivariate anomaly detection for time series data with generative adversarial networks. In *International Conference on Artificial Neural Networks*, pages 703–716, 2019.

[70] Yan Li, Oceane Bel, Kenneth Chang, Ethan L. Miller, and Darrell D. E. Long. CAPES: Unsupervised storage performance tuning using neural network-based deep reinforcement learning. In *Proceedings of the 2015 International Conference for High Performance Computing, Networking, Storage and Analysis (SC17)*, November 2017.

[71] Yan Li, Yash Gupta, Ethan L. Miller, and Darrell D. E. Long. Pilot: A framework that understands how to do performance benchmarks the right way. In *Proceedings of the 24th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS '16)*, September 2016.

[72] Yan Li and Darrell D. E. Long. Which storage device is the greenest? Modeling the energy cost of I/O workloads. In *Proceedings of the 22nd International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS '14)*, 2014.

[73] Yan Li, Xiaoyuan Lu, Ethan L. Miller, and Darrell D. E. Long. The ASCAR open source project. `http://www.ssrc.ucsc.edu/ascar.html`, 2015.

[74] N. Markatchev and C. Williamson. WebTraff: a GUI for web proxy cache workload modeling and analysis. In *Proceedings. 10th IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunications Systems*, pages 356–363. IEEE, Oct. 2002.

[75] Richard McDougall and Jim Mauro. Filebench. `http://www.nfsv4bat.org/Documents/nasconf/2004/filebench.pdf`, 2005.

[76] Pankaj Mehra and Benjamin W. Was. Physical-level synthetic workload generation for load-balancing experiments. In *Proceedings of the First International Symposium on High-Performance Distributed Computing*, pages 208–217. IEEE, Sept. 1992.

[77] Michael Minelli, Michele Chambers, and Ambiga Dhiraj. *Big data, big analytics: emerging business intelligence and analytic trends for today's businesses*, volume 578. John Wiley & Sons, 2013.

[78] Alberto Miranda, Sascha Effert, Yangwook Kang, Ethan L. Miller, Ivan Popov, André Brinkmann, Tom Friedetzky, and Toni Cortes. Random Slicing: Efficient and scalable data placement for large-scale storage systems. *ACM Transactions on Storage*, 10(3), July 2014.

[79] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. *ArXiv*, abs/1411.1784, Nov. 2014.

[80] Sinjoni Mukhopadhyay, Joel Frank, Justin King, Daniel Bittman, Darrell Long, and Ethan Miller. Efficient reconstruction techniques for disaster recovery in secret-split datastores. In *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pages 66–78, 2018.

[81] Computational NCAR Research Data Archive and Information Systems Lab. Ncar data archive. `https://rda.ucar.edu`, 2019.

[82] Tim Nufire. Petabytes on a budget: How to build cheap cloud storage. https://www.backblaze.com/blog/petabytes-on-a-budget-how-to-build-cheap-cloud-storage/, 2009. Date Retrieved 09/01/2009.

[83] Augustus Odena, Christopher Olah, and Jonathon Shlens. Conditional image synthesis with auxiliary classifier gans. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, pages 2642–2651. JMLR.org, 2017.

[84] Jehan-François Pâris, Thomas Schwarz, Ahmed Amer, and Darrell D. E. Long. Protecting RAID arrays against unexpectedly high disk failure rates. In *Proceedings of the IEEE 20th Pacific Rim International Symposium on Dependable Computing (PRDC '14)*, pages 68–75, Nov 2014.

[85] Jehan-François Pâris, Ahmed Amer, Darrell D. E. Long, and Thomas J. E. Schwarz. Self-repairing disk arrays. In *Proceedings of the 5th International Workshop on Adaptive Self-tuning Computing Systems 2015 (ADAPT'15)*, 2015.

[86] Noseong Park, Mahmoud Mohammadi, Kshitij Gorde, Sushil Jajodia, Hongkyu Park, and Youngmin Kim. Data synthesis based on generative adversarial networks. *Proceedings of the VLDB Endowment*, 11(10):1071–1083, June 2018.

[87] Aleatha Parker-Wood, Darrell D. E. Long, Ethan L. Miller, Philippe Rigaux, and Andy Isaacson. A file by any other name: Managing file names with metadata. In *Proceedings of the 7th Annual International Systems and Storage Conference (SYSTOR 2014)*, June 2014.

[88] Xue Bin Peng, Angjoo Kanazawa, Sam Toyer, Pieter Abbeel, and Sergey Levine. Variational discriminator bottleneck: Improving imitation learning, inverse rl, and gans by constraining information flow. *ArXiv*, abs/1810.00821, Oct. 2018.

[89] Rekha Pitchumani, Shayna Frank, and Ethan L. Miller. Realistic request arrival generation in storage benchmarks. In *Proceedings of the 31st IEEE Conference on Mass Storage Systems and Technologies*, June 2015.

[90] Rekha Pitchumani, James Hughes, and Ethan L. Miller. SMRDB: Key-value data store for shingled magnetic recording disks. In *Proceedings of the 8th Annual International Systems and Storage Conference (SYSTOR 2015)*, May 2015.

[91] SNIA Technical Position. Real world storage workload (rwsw) performance test specification for datacenter storage. https://www.snia.org/sites/default/files/technical_work/RWSW/RWSW_Spec_v1.0.7.pdf, 2018. Date Retrieved 05/25/2018.

[92] Sebastien Razakarivony and Frédéric Jurie. Discriminative autoencoders for small targets detection. In *2014 22nd International Conference on Pattern Recognition*, pages 3528–3533, Aug. 2014.

[93] Jinglei Ren, Jishen Zhao, Samira Khan, Jongmoo Choi, Yongwei Wu, and Onur Mutlu. ThyNVM: Enabling software-transparent crash consistency in persistent memory systems. In

Proceedings of the International Symposium on Microarchitecture (MICRO'15), December 2015.

[94] SNIA Iotta Repository. Downloading snia traces. `http://iotta.snia.org/traces/`, 2008.

[95] Andrea Paz Rodriguez Delherbe. CERNBox logs ingestion and analysis. Technical Report CERN-STUDENTS-Note-2019-107, CERN, Geneva, Aug 2019.

[96] Ruslan Salakhutdinov, Andriy Mnih, and Geoffrey Hinton. Restricted boltzmann machines for collaborative filtering. In *Proceedings of the 24th international conference on Machine learning*, pages 791–798, 2007.

[97] Milad Salem, Shayan Taheri, and Jiann Shiun Yuan. Anomaly generation using generative adversarial networks in host based intrusion detection. *arXiv preprint arXiv:1812.04697*, 2018.

[98] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. In *Advances in neural information processing systems*, pages 2234–2242, 2016.

[99] Soumalya Sarkar, Kin G Lore, Soumik Sarkar, Vikram Ramanan, Satyanarayanan R Chakravarthy, Shashi Phoha, and Asok Ray. Early detection of combustion instability from hi-speed flame images via deep learning and symbolic time series analysis. In *Annual Conf. of the Prognostics and Health Management*, volume 6, 2015.

[100] Konstantinos Amplianitis Sebastian Lutz and Aljosa Smolic. AlphaGAN: Generative adversarial networks for natural image matting. *ArXiv*, abs/1807.10088, July 2018.

[101] Jeff Sedayao, Rahul Bhardwaj, and Nakul Gorade. Making big data, privacy, and anonymization work together in the enterprise: experiences and issues. In *2014 IEEE International Congress on Big Data*, pages 601–607, June 2014.

[102] Martin Simonovsky and Nikos Komodakis. Graphvae: Towards generation of small graphs using variational autoencoders. In *International Conference on Artificial Neural Networks*, pages 412–422, 2018.

[103] Pawan Sinha and Richard Russell. A perceptually based comparison of image similarity metrics. *Perception*, 40(11):1269–1281, Nov. 2011.

[104] Maximilian Sölch, Justin Bayer, Marvin Ludersdorfer, and Patrick van der Smagt. Variational inference for on-line anomaly detection in high-dimensional time series. *ArXiv*, abs/1602.07109, Feb. 2016.

[105] Adrian Spurr, Emre Aksan, and Otmar Hilliges. Guiding infogan with semi-supervision. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 119–134, 2017.

[106] Kostas G.Anagnostakis Spyros Antonatos and Evangelos P.Markatos. Generating realistic workloads for network intrusion detection systems. In *Proceedings of the 4th international workshop on Software and performance*, pages 207–215. ACM, 2004.

[107] Luke Stone. Bringing pokémon go to life on google cloud. `https://cloud.google.com/blog/products/gcp/bringing-pokemon-go-to-life-on-google-cloud`. Date Retrieved 09/29/2016.

[108] Shuntaro Takahashi, Yu Chen, and Kumiko Tanaka-Ishii. Modeling financial time-series with generative adversarial networks. *Physica A: Statistical Mechanics and its Applications*, 527:121261, Aug. 2019.

[109] Wei Ren Tan, Chee Seng Chan, Hernán E Aguirre, and Kiyoshi Tanaka. Artgan: Artwork synthesis with conditional categorical gans. In *2017 IEEE International Conference on Image Processing (ICIP)*, pages 3760–3764. IEEE, Sept. 2017.

[110] Andrew S Tanenbaum and Albert S Woodhull. *Operating systems: design and implementation*, volume 68. Prentice Hall Englewood Cliffs, 1997.

[111] Graham W. Taylor, Geoffrey E Hinton, and Sam T. Roweis. Modeling human motion using binary latent variables. In B. Schölkopf, J. C. Platt, and T. Hoffman, editors, *Advances in Neural Information Processing Systems 19*, pages 1345–1352, 2007.

[112] Graham W Taylor, Geoffrey E Hinton, and Sam T Roweis. Two distributed-state models for generating high-dimensional time series. *Journal of Machine Learning Research*, 12:1025–1068, Mar 2011.

[113] Wasabi Technologies. Wasabi performance benchmark report. `https://i.crn.com/sites/default/files/ckfinderimages/userfiles/images/crn/custom/Wasabi_Performance_Benchmark_Report-18.pdf`, 2018.

[114] Erich Thanhardt and Gene Harano. File migration in the ncar mass storage system. In *Digest of Papers Ninth IEEE Symposium on Mass Storage Systems, 1988.'Storage Systems: Perspectives'*, pages 114–121, 1988.

[115] Lincoln Thurlow, Andrew Kwong, Thomas J. E. Schwarz, and Ethan L. Miller. gferasure: a high performance Galois field library for erasure coding and algebraic signature computation. `https://bitbucket.org/ssrc/gferasure`.

[116] Thomas Unterthiner, Bernhard Nessler, Calvin Seward, Günter Klambauer, Martin Heusel, Hubert Ramsauer, and Sepp Hochreiter. Coulomb gans: Provably optimal nash equilibria via potential fields. *ArXiv*, abs/1708.08819, Aug. 2017.

[117] Michele C. Weigle, Prashanth Adurthi, Félix Hernández-Campos, Kevin Jeffay, and F. Donelson Smith. Tmix: A tool for generating realistic tcp application workloads in ns-2. *SIGCOMM Comput. Commun. Rev.*, 36(3):65–76, July 2006.

[118] Ludmila Cherkasova Wenting Tang, Yan Fu and Amin Vahdat. MediSyn: A synthetic streaming media service workload generator. In *Proceedings of the 13th international workshop on Network and operating systems support for digital audio and video*, pages 12–21. ACM, 2003.

[119] Avani Wildani, Ian F. Adams, and Ethan L. Miller. Single-snapshot file system analysis. In *Proceedings of the 21st International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS '13)*, August 2013.

[120] Avani Wildani and Ethan L. Miller. Can we group storage? statistical techniques to identify predictive groupings in storage system accesses. *ACM Transactions on Storage*, 12(2), February 2016.

[121] Avani Wildani, Ethan L. Miller, Ian Adams, and Darrell D. E. Long. PERSES: Data layout for low impact failures. In *Proceedings of the 22nd International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS '14)*, September 2014.

[122] Charles P Wright, Nikolai Joukov, Devaki Kulkarni, Yevgeniy Miretskiy, and Erez Zadok. Auto-pilot: A platform for system software benchmarking. In *USENIX Annual Technical Conference, FREENIX Track*, pages 175–188, Apr 2005.

[123] Qingtao Wu and Zhiqing Shao. Network anomaly detection using time series analysis. In *Joint International Conference on Autonomic and Autonomous Systems and International Conference on Networking and Services-(icas-isns' 05)*, pages 42–42. IEEE, 2005.

[124] Tao Xu, Pengchuan Zhang, Qiuyuan Huang, Han Zhang, Zhe Gan, Xiaolei Huang, and Xiaodong He. Attngan: Fine-grained text to image generation with attentional generative adversarial networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1316–1324, 2018.

[125] Chi Zhang, Sanmukh R Kuppannagari, Rajgopal Kannan, and Viktor K Prasanna. Generative adversarial network for synthetic time series data generation in smart grids. In *2018 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, pages 1–6, 2018.

[126] Chunkai Zhang and Yingyang Chen. Time series anomaly detection with variational autoencoders. *ArXiv*, abs/1907.01702, July 2019.

[127] Bin Zhou, Shenghua Liu, Bryan Hooi, Xueqi Cheng, and Jing Ye. Beatgan: Anomalous rhythm detection using adversarially generated time series. In *International Joint Conference on Artificial Intellince (IJCAI)*, pages 4433–4439, July 2019.